

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON (8 SEMANAS)

NIVEL 3 (AVANZADO)

Curso 3: Creación y Desarrollo de Agentes de IA

«Modelos LLM, Inferencia, Tool Calling, Memoria Vectorial, RAG, Arquitecturas ReAct y Multi-Agentes»

Manual completo del curso compilado oficialmente. Incluye el desglose de las 8 clases semanales, diagramas de arquitectura, código fuente comentado, resolución de gotchas y bibliografía de estudio.

Instructor: William Rodríguez (Wisrovi)

Total Clases: 8 Semanas | Python: 3.10+ | Licencia: MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología: La Regla de la Bicicleta

Aprender a programar es pedalear en VS Code: la experimentación práctica diaria es la clave del dominio técnico.

Plan de Estudios de las 8 Semanas

Semana	Unidad Temática	Metáfora
CLASE 01	Clase 01: Fundamentos de LLMs, Tokens y Arquitectura Transformer	«Modelos de Lenguaje como Motores de Predicción Pro...»
CLASE 02	Clase 02: Prompt Engineering Avanzado y Few-Shot Learning	«Prompts como Especificaciones Precisas para un Con...»
CLASE 03	Clase 03: Salidas Estructuradas y Validación Tipada con Pydantic V2	«Pydantic como la Aduana Estricta de Datos para Res...»
CLASE 04	Clase 04: Tool Calling y Function Calling en Python	«Dotando de Manos y Herramientas al Cerebro del LLM...»
CLASE 05	Clase 05: Embeddings y Representación Vectorial Semántica	«Embeddings como Coordenadas GPS del Significado de...»
CLASE 06	Clase 06: Arquitecturas RAG (Retrieval-Augmented Generation)	«RAG como Darle al LLM un Libro Abierto con la Info...»
CLASE 07	Clase 07: Agentes Autónomos y el Ciclo Cognitivo ReAct	«El Agente como un Detective que Piensa, Actúa y Ob...»
CLASE 08	Clase 08: Sistemas Multi-Agente, Supervisión y Guardrails	«Una Empresa de Agentes Especializados Coordinados ...»

Clase 01: Fundamentos de LLMs, Tokens y Arquitectura Transformer

Los Modelos de Lenguaje Grande (LLMs) son redes neuronales basadas en la arquitectura Transformer que predicen el siguiente token.

☀ **Metáfora: Modelos de Lenguaje como Motores de Predicción Probabilística**

Un LLM es como el teclado predictivo de tu móvil, pero entrenado con todo el conocimiento digital del planeta.

Principios Teóricos

Los LLMs no procesan palabras ni letras, procesan 'tokens' (fragmentos de palabras de ~4 caracteres).

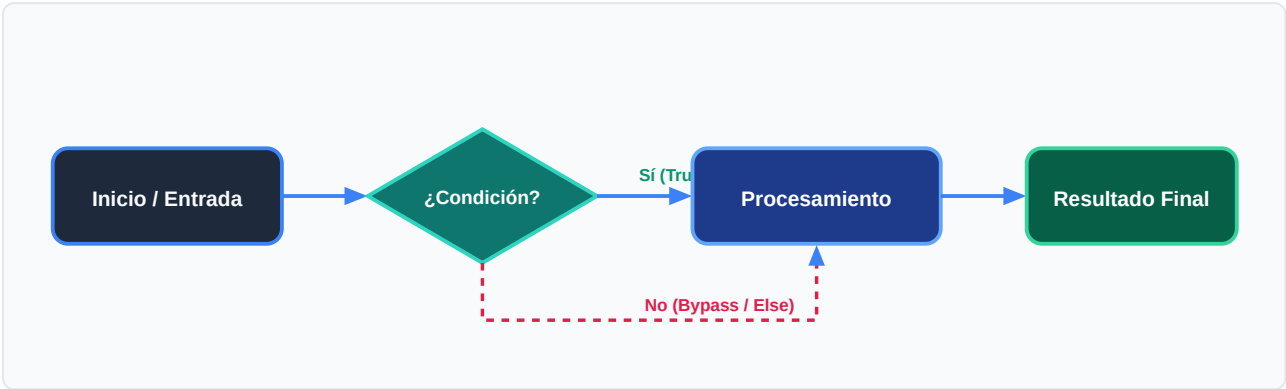
La temperatura controla la entropía de la distribución probabilística (0.0 = determinista, 1.0 = creativo).

⚡ **Regla de Oro**

Para tareas de extracción estructurada, código o datos, mantén siempre la temperatura en 0.0.

Arquitectura y Movimiento de Datos

Ciclo de tokenización, atención Transformer y muestreo de salida.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Conversión de texto a lista de enteros (Token IDs).	Vector de tokens de entrada.
2. Proceso	Paso hacia adelante en capas de auto-atención.	Distribución de Logits calculada.
3. Salida	Decodificación de token a texto y streaming.	Respuesta generada token por token.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
def simular_tokenizador(texto: str) -> list[str]:  
    # Simulación básica de subwords  
    return texto.replace(".", " .").split()  
  
tokens = simular_tokenizador("Python es el lenguaje líder en Inteligencia Artificial.")  
print(f"Total tokens: {len(tokens)}")  
print("Tokens extraídos:", tokens)
```

⚠ Gotcha Frecuente

Enviar documentos gigantes sin podar agota la ventana de contexto y dispara los costos de tokens.

❌ Antipatrón

```
prompt = doc_entero_de_500_paginas + '\nResume esto' #  
❌ Desborda el contexto
```

✅ Patrón Correcto

```
# Chunking previo y filtrado semántico RAG ✅
```

Clase 02: Prompt Engineering Avanzado y Few-Shot Learning

El Prompt Engineering es la disciplina de diseñar entradas estructuradas para guiar a los LLMs hacia resultados precisos.

☀ Metáfora: Prompts como Especificaciones Precisas para un Consultor Experto

El System Prompt es como el contrato de trabajo de un empleado: define su rol, límites, tono y reglas inquebrantables.

Principios Teóricos

Zero-Shot: Instrucción directa sin ejemplos previos.

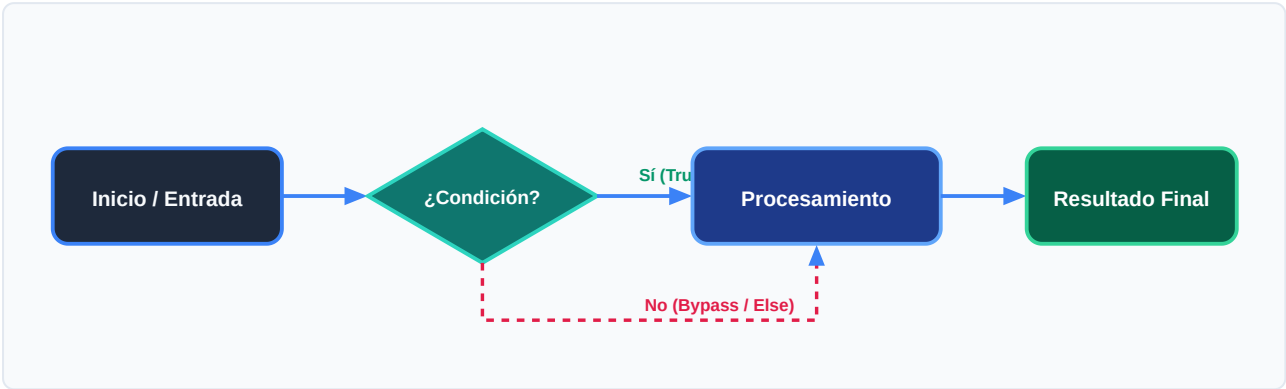
Few-Shot In-Context Learning: Proporcionar de 2 a 5 ejemplos de pares entrada-salida para fijar el patrón de respuesta.

⚡ Regla de Oro

Instruye al modelo sobre lo que DEBE hacer, en lugar de solo listar lo que no debe hacer.

Arquitectura y Movimiento de Datos

Estructura de mensajes por roles y razonamiento paso a paso (CoT).



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Definición del System Prompt con restricciones de seguridad.	Rol y contexto fijados.
2. Proceso	Inclusión de ejemplos Few-Shot formateados.	Patrón de inferencia condicionado.
3. Salida	Generación de respuesta final concisa.	Salida alineada con el formato esperado.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
TEMPLATE_SYSTEM = """Eres un clasificador de soporte técnico. Responde ÚNICAMENTE en formato JSON.
Roles permitidos de sentimiento: POSITIVO, NEGATIVO, NEUTRO."""

EJEMPLOS_FEW_SHOT = [
    {"input": "La app se cierra sola", "output": '{"sentimiento": "NEGATIVO", "urgencia": "ALTA"}'},
    {"input": "Excelente servicio y soporte", "output": '{"sentimiento": "POSITIVO", "urgencia": "BAJA"}'}
]

def construir_prompt(consulta_usuario: str) -> str:
    return f"{TEMPLATE_SYSTEM}\n\nEjemplos:\n{EJEMPLOS_FEW_SHOT}\n\nUsuario: {consulta_usuario}"

print(construir_prompt("No puedo iniciar sesión"))
```

⚠ Gotcha Frecuente

Concatenar texto de usuarios sin sanitizar permite que instrucciones maliciosas anulen el System Prompt.

✗ Antipatrón

```
prompt = f'Eres un bot. Traduce: {input_usuario}' # ✗
Si el usuario pone 'Olvida las reglas anteriores...',
el bot obedece
```

✓ Patrón Correcto

```
# Uso de delimitadores XML y guardrails de validación
✓
```

Clase 03: Salidas Estructuradas y Validación Tipada con Pydantic V2

Integrar LLMs en sistemas empresariales exige que sus respuestas sean 100% deterministas en estructura y tipo.

☀ Metáfora: Pydantic como la Aduana Estricta de Datos para Respuestas de IA

Pydantic es el inspector de aduana que revisa que cada paquete traiga exactamente los sellos, tipos y formatos requeridos.

Principios Teóricos

Pydantic V2 está construido sobre un núcleo en Rust de alto rendimiento para validación instantánea.

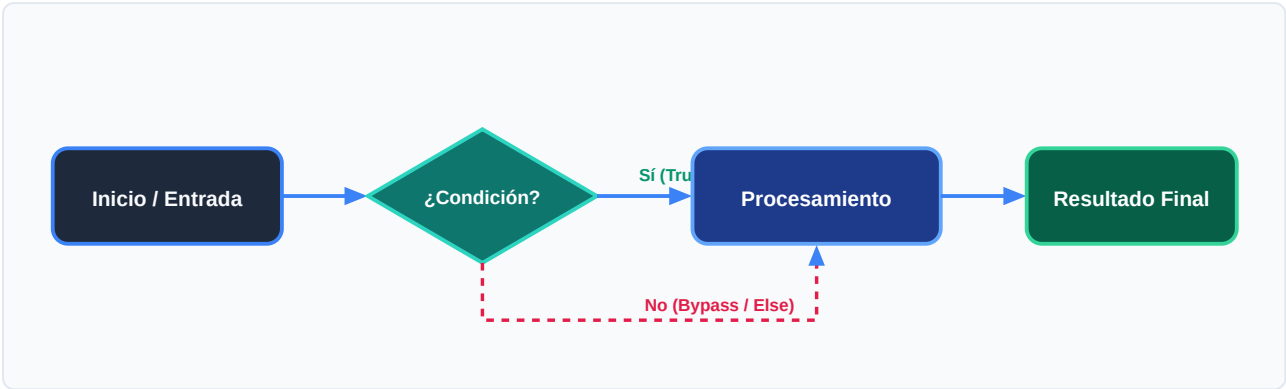
Structured Outputs obliga al LLM a seguir la gramática JSON Schema del modelo BaseModel.

⚡ Regla de Oro

Nunca consumas texto libre de un LLM en lógica transaccional; valida siempre con Pydantic.

Arquitectura y Movimiento de Datos

Flujo de inferencia con esquema JSON y validación Pydantic.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Definición del modelo BaseModel con campos y validadores Field().	Esquema JSON Schema exportado.
2. Proceso	Inyección del esquema en la llamada de API del LLM.	Inferencia restringida por gramática.
3. Salida	Validación con Modelo.model_validate_json().	Objeto Python fuertemente tipado instanciado.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
from pydantic import BaseModel, Field, EmailStr

class LeadCliente(BaseModel):
    nombre: str = Field(description="Nombre completo del prospecto")
    email: str = Field(description="Correo electrónico válido")
    presupuesto_estimado: float = Field(ge=0.0, description="Monto en USD")
    interes_ia: bool = True

# Simulación de respuesta JSON generada por LLM
json_llm = '{"nombre": "Laura Méndez", "email": "laura@empresa.com", "presupuesto_estimado": 15000.0}'
lead = LeadCliente.model_validate_json(json_llm)

print("Lead Validado:", lead.nombre)
print("Presupuesto:", lead.presupuesto_estimado)
```

⚠ Gotcha Frecuente

Parsear la salida del LLM con `json.loads()` simple sin validar tipos permite que campos nulos rompan la aplicación.

❌ Antipatrón

```
data = json.loads(respuesta_llm) total = data['precio']
* 2 # ❌ Falla si 'precio' vino como None o string
```

✅ Patrón Correcto

```
data = FacturaModel.model_validate_json(respuesta_llm)
total = data.precio * 2 # ✅ Garantizado float tipado
```

Clase 04: Tool Calling y Function Calling en Python

Tool Calling permite que un LLM decida autónomamente cuándo invocar funciones de código externo para consultar datos o actuar.

☀ Metáfora: Dotando de Manos y Herramientas al Cerebro del LLM

El LLM es un cerebro brillante pero ciego y sin manos; las herramientas son sus brazos mecánicos para interactuar con el mundo.

Principios Teóricos

El LLM NO ejecuta el código directamente: devuelve un objeto estructurado con el nombre de la función y sus argumentos.

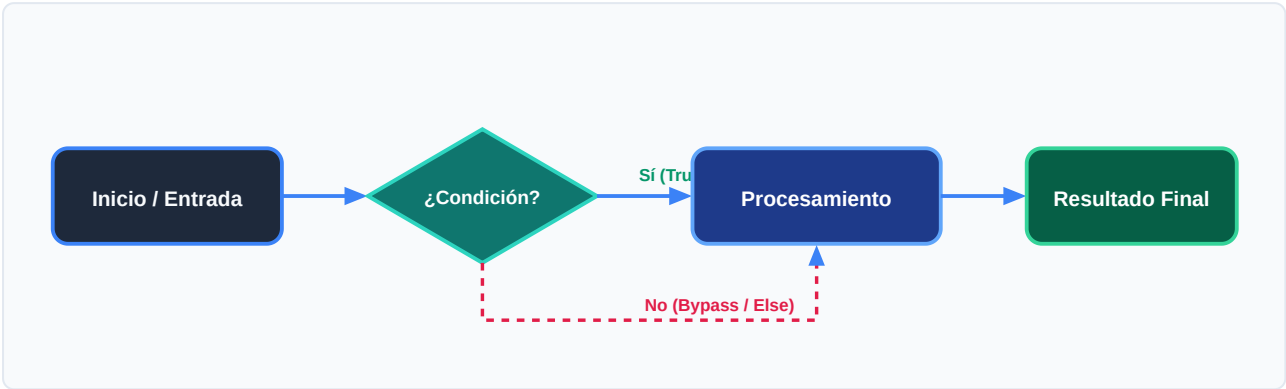
Tu backend intercepta la solicitud, ejecuta la función real en Python y le devuelve el resultado al LLM.

⚡ Regla de Oro

Escribe docstrings extremadamente claros en tus funciones: el LLM los usa como manual de instrucciones.

Arquitectura y Movimiento de Datos

Ciclo completo de Tool Calling: Solicitud -> Despacho -> Ejecución -> Respuesta.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Registro de funciones en el catálogo de herramientas.	Definiciones de herramientas cargadas.
2. Proceso	El LLM analiza el prompt y genera un ToolCall con argumentos.	Llamada pausada esperando ejecución.
3. Salida	Envío del resultado de vuelta al LLM para formular la respuesta final.	Respuesta natural entregada al usuario.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
import math

def calcular_distancia(x1: float, y1: float, x2: float, y2: float) -> float:
    """Calcula la distancia euclidiana entre dos puntos (x1, y1) y (x2, y2)."""
    return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

HERRAMIENTAS = {
    "calcular_distancia": calcular_distancia
}

def despachar_herramienta(nombre: str, argumentos: dict):
    if nombre in HERRAMIENTAS:
        return HERRAMIENTAS[nombre](**argumentos)
    raise ValueError(f"Herramienta '{nombre}' no encontrada.")

res = despachar_herramienta("calcular_distancia", {"x1": 0.0, "y1": 0.0, "x2": 3.0, "y2": 4.0})
print("Resultado de la herramienta:", res) # 5.0
```

⚠ Gotcha Frecuente

Usar `eval()` o `exec()` para ejecutar herramientas abre una vulnerabilidad crítica de inyección de código.

❌ Antipatrón

```
eval(f'{nombre_funcion}({argumentos_crudos})') # ❌
```

Vulnerabilidad RCE crítica

✅ Patrón Correcto

```
HERRAMIENTAS[nombre](**argumentos) # ✅
```

Mapeo explícito a funciones seguras

Clase 05: Embeddings y Representación Vectorial Semántica

Los embeddings transforman texto en vectores de números que capturan el significado semántico y contextual.

☀ Metáfora: Embeddings como Coordenadas GPS del Significado de las Palabras

Un embedding es como la latitud y longitud de un concepto: 'Rey' y 'Reina' están muy cerca en el mapa semántico.

Principios Teóricos

Textos con significados similares tienen vectores que apuntan en direcciones casi idénticas en el espacio n-dimensional.

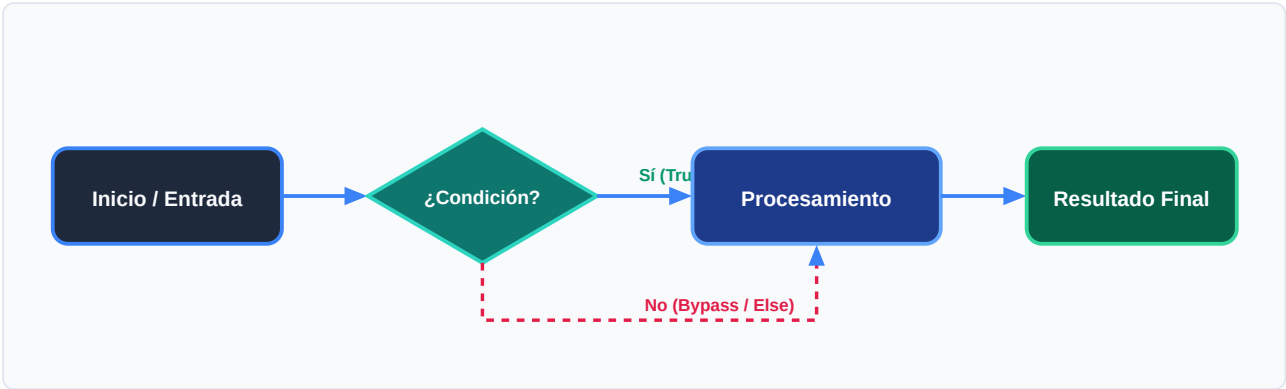
La Similitud de Coseno mide el coseno del ángulo entre dos vectores (1.0 = idénticos, 0.0 = ortogonales).

⚡ Regla de Oro

Los embeddings permiten búsquedas por SIGNIFICADO, no solo por coincidencia exacta de palabras clave.

Arquitectura y Movimiento de Datos

Transformación de texto a vector y cálculo de similitud espacial.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Paso del texto por el modelo de embedding.	Vector denso de floats (ej. 768 dimensiones).
2. Proceso	Almacenamiento del vector e indexación.	Espacio vectorial poblado.
3. Salida	Ordenamiento por ranking de relevancia.	Top K documentos más cercanos.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
import math

def similitud_coseno(v1: list[float], v2: list[float]) -> float:
    dot_product = sum(a * b for a, b in zip(v1, v2))
    norm_v1 = math.sqrt(sum(a * a for a in v1))
    norm_v2 = math.sqrt(sum(b * b for b in v2))
    if norm_v1 == 0 or norm_v2 == 0: return 0.0
    return dot_product / (norm_v1 * norm_v2)

# Vectores conceptuales simulados
vec_python = [0.9, 0.8, 0.1]
vec_codigo = [0.85, 0.75, 0.15]
vec_cocina = [0.05, 0.1, 0.95]

print("Similitud Python vs Código:", round(similitud_coseno(vec_python, vec_codigo), 4))
print("Similitud Python vs Cocina:", round(similitud_coseno(vec_python, vec_cocina), 4))
```

⚠ Gotcha Frecuente

Comparar embeddings generados por dos modelos distintos produce resultados erróneos.

❌ Antipatrón

```
similitud(emb_openai_1536, emb_bge_768) # ❌
```

Incompatibilidad de dimensiones

✅ Patrón Correcto

```
# Usa SIEMPRE el mismo modelo de embedding para indexar y consultar ✅
```

Clase 06: Arquitecturas RAG (Retrieval-Augmented Generation)

RAG combina recuperación semántica con generación de lenguaje para que el LLM responda con datos privados sin reentrenamiento.

☀ **Metáfora: RAG como Darle al LLM un Libro Abierto con la Información Exacta**

En lugar de pedirle al alumno que responda de memoria, le permites consultar el capítulo exacto del libro antes de contestar.

Principios Teóricos

Ingestión: Carga de documentos -> División en Chunks (ej. 500 caracteres con 50 de solapamiento) -> Embeddings -> Vector Store.

Recuperación y Aumento: Consulta de usuario -> Búsqueda vectorial Top-K -> Inyección en el System Prompt.

⚡ **Regla de Oro**

Instruye al modelo a responder ÚNICAMENTE basándose en el contexto provisto para eliminar alucinaciones.

Arquitectura y Movimiento de Datos

Pipeline RAG de 2 fases: Ingestión Indexada y Consulta en Tiempo Real.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Chunking de documentos y generación de embeddings.	Vectores almacenados en el Vector Store.
2. Proceso	Recepción de pregunta del usuario y embedding de la query.	Vector de consulta generado.
3. Salida	Construcción del prompt aumentado y generación final del LLM.	Respuesta precisa con citas de fuentes.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
class MiniRAG:
    def __init__(self):
        self.docs = []

    def indexar(self, texto: str):
        # Simulación de chunking básico
        self.docs.append(texto)

    def recuperar(self, query: str) -> str:
        # Recupera el documento con mayor coincidencia léxica
        palabras = set(query.lower().split())
        mejor_doc = max(self.docs, key=lambda d:
            len(palabras.intersection(set(d.lower().split()))))
        return mejor_doc

    def generar_prompt(self, query: str) -> str:
        ctx = self.recuperar(query)
        return f"Contexto:\n{ctx}\n\nPregunta: {query}\nRespuesta basada estrictamente en el contexto:"

rag = MiniRAG()
rag.indexar("El horario de atención es de Lunes a Viernes de 9:00 a 18:00.")
print(rag.generar_prompt("¿A qué hora abren?"))
```

⚠ Gotcha Frecuente

Inyectar demasiados chunks (ej. 20 chunks) satura el contexto y hace que el LLM ignore la información central.

❌ Antipatrón

```
prompt = f'Contexto:\n{20_chunks_desordenados}' # ❌
```

Degradación de atención

✅ Patrón Correcto

```
# Selecciona Top 3 a 5 chunks relevantes y reordénalos con un Re-ranker ✅
```

Clase 07: Agentes Autónomos y el Ciclo Cognitivo ReAct

Un Agente de IA es un sistema autónomo que combina un LLM con memoria, herramientas y un bucle de razonamiento para alcanzar metas.

☀ **Metáfora: El Agente como un Detective que Piensa, Actúa y Observa hasta Resolver el Caso**

Un agente es como un detective: piensa qué pista necesita (Thought), busca el dato con una herramienta (Action), analiza el resultado (Observation) y repite.

Principios Teóricos

Ciclo ReAct: Thought (Pensamiento) -> Action (Acción/Tool) -> Observation (Resultado) -> Final Answer.

El agente evalúa si la observación resuelve el objetivo o si requiere ejecutar acciones adicionales.

⚡ **Regla de Oro**

Establece siempre un 'max_iterations = 5' para evitar que el agente quede atrapado en bucles infinitos.

Arquitectura y Movimiento de Datos

Bucle autónomo de decisión ReAct con parada por condición de meta.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Recepción de la meta del usuario.	Objetivo fijado en memoria de trabajo.
2. Proceso	Generación de Thought y selección de Action.	Herramienta elegida con parámetros.
3. Salida	¿Meta alcanzada? Sí -> Final Answer / No -> Siguiente ciclo.	Respuesta final entregada.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
class ReActAgent:
    def __init__(self, tools: dict, max_steps: int = 3):
        self.tools = tools
        self.max_steps = max_steps
        self.memory = []

    def run(self, goal: str):
        print(f"🎯 Meta: {goal}")
        for step in range(1, self.max_steps + 1):
            print(f"--- Paso {step} ---")
            # 1. Thought
            thought = f"Necesito consultar la cotización del euro."
            print(f"💭 Thought: {thought}")

            # 2. Action
            obs = self.tools["get_rate"]("EUR_USD")
            print(f"📜 Action: get_rate(EUR_USD) -> Obs: {obs}")

            # 3. Final Answer
            return f"Respuesta Final: 1 EUR equivale a {obs} USD."

tools = {"get_rate": lambda pair: 1.08}
agente = ReActAgent(tools)
print(agente.run("¿Cuánto vale el euro frente al dólar?"))
```

⚠️ Gotcha Frecuente

Si una herramienta falla o devuelve error, el agente puede reintentar la misma acción en un ciclo sin fin.

❌ Antipatrón

`while not finished: agent.step()` # ❌ Puede consumir tokens infinitos

✅ Patrón Correcto

`for step in range(max_steps):` # ✅ Límite estricto de seguridad

Clase 08: Sistemas Multi-Agente, Supervisión y Guardrails

Para problemas complejos, múltiples agentes especializados colaboran mejor que un único agente generalista.

☀ Metáfora: Una Empresa de Agentes Especializados Coordinados por un Director

Es como una agencia de noticias: el reportero investiga los hechos, el redactor escribe la noticia y el editor jefe revisa la calidad.

Principios Teóricos

Patrón Supervisor: Un agente orquestador recibe la tarea global y la desglosa delegando a agentes especialistas.

Guardrails: Filtros deterministas que validan entradas y salidas para prevenir toxicidad, fugas de datos (PII) y alucinaciones.

⚡ Regla de Oro

Asigna a cada agente un System Prompt ultra específico y un conjunto reducido de herramientas.

Arquitectura y Movimiento de Datos

Orquestación multi-agente con supervisor y validación por guardrails.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Recepción de solicitud y análisis del Supervisor.	Plan de tareas desglosado.
2. Proceso	Delegación al Agente Investigador (Tool Calling).	Datos crudos recolectados.
3. Salida	Filtro por Guardrails de seguridad y aprobación final.	Entrega final aprobada.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
class MultiAgentSystem:
    def __init__(self):
        pass

    def agente_investigador(self, tema: str) -> dict:
        return {"datos": f"Hallazgos clave sobre {tema}: Crecimiento del 40% en adopción."}

    def agente_redactor(self, investigacion: dict) -> str:
        return f"Reporte Ejecutivo: {investigacion['datos']}"

    def supervisor(self, tema: str) -> str:
        print("👑 Supervisor: Coordinando equipo...")
        datos = self.agente_investigador(tema)
        informe = self.agente_redactor(datos)
        return f"✅ Publicación Aprobada:\n{informe}"

sistema = MultiAgentSystem()
print(sistema.supervisor("Agentes Autónomos en 2026"))
```

⚠️ Gotcha Frecuente

Pasar texto libre desordenado entre agentes provoca pérdida de contexto en cadenas largas.

❌ Antipatrón

```
msg_agente_2 = call_llm(f'El otro dijo:
{texto_libre_caotico}') # ❌ Degradación
```

✅ Patrón Correcto

```
# Usa esquemas Pydantic para el paso de mensajes entre
agentes ✅
```

Conclusiones Generales del Curso

Has completado las 8 semanas de **Curso 3: Creación y Desarrollo de Agentes de IA**.



Certificación del Nivel

Dominio conceptual y práctico consolidado.

Bibliografía Canónica

Recurso	Descripción	Enlace
Documentación Oficial Python	Especificación canónica	docs.python.org/3/
PEP 8	Guía de estilo	peps.python.org/pep-0008/
wisrovi SUITE	Librerías open source	github.com/wisrovi